SUBMISSION INSTRUCTIONS

It is recommendend that you make a copy of this colab file and then solve the assignment and upload your final notebook on github.

Before uploading your downloaded notebook, **RENAME** the file as **rollno_name.ipynb**

Submission Deadline : 9/12/2025 Tuesday EOD i.e before 11:59 PM

The deadline is strict and will not be extended, Late submissions are not allowed

Note that you have to upload your solution on the github page of the project Vision Transformer and under Week0

Github Submission repo - https://github.com/electricalengineersiitk/Winter-projects-25-26/tree/main/Vision%20transformer/Week0

Assignment 1

Section 1 (Python)

Problem 1

In this problem, you will implement a class that represents a data sample with numerical features.

Problem Statement

Create a class named `DataSample` that stores:

- A list of numeric **features**
- A string **label**

You must implement the following **three methods** with **exact names** (do NOT rename them):

Method	Description
`__init__(self, features, label)`	Initializes object attributes
`min_max_norm(self)`	Apply min-max normalization **in-place**
`scaled(self, factor)`	Return a **new list** with each feature multiplied by `factor`

Min–Max Normalization Formula

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Edge Case

If **all features are equal**, then max = min → division by zero.
Handle this condition by setting all normalized values to **0**.

Your output must behave conceptually like this (not real execution here):

```
sample = DataSample([10, 20, 30], "cat")
sample.min_max_norm()
print(sample.features)   # expected -> [0.0, 0.5, 1.0]

print(sample.scaled(2))  # expected -> [0.0, 1.0, 2.0]
```

Sample Class

```
class DataSample:
    def __init__(self, features, label):
        """
        Initialize the object with feature list and label string.
        """
        pass

    def min_max_norm(self):
        """
```

```
        Apply min-max normalization:
        Modify self.features in-place.
        Handle case where max == min.
        """
        pass

    def scaled(self, factor):
        """
        Return a NEW list of features where each element is multiplied by factor.
        """
        pass
```

You can check if your code is working correctly using the sample case below

```
sample = DataSample([3.5, -2.0, 3.5, 10.0, 0.0], "bird")

print("Original features:", sample.features)
print("Label:", sample.label)

sample.min_max_norm()
print("After min-max normalization:", sample.features)

scaled_output = sample.scaled(4.2)
print("After scaling x 4.2 (new list expected):", scaled_output)

# Expected Output:
# Original features: [3.5, -2.0, 3.5, 10.0, 0.0]
# Label: bird
# After min-max normalization: [0.4583, 0.0, 0.4583, 1.0, 0.1667]  (approx values)
# After scaling x 4.2: [1.92486, 0.0, 1.92486, 4.2, 0.70014]
```

Problem 2

Sort Based on Unique Character Count

Write a function named `sort_by_unique_chars` that sorts a list of strings based on the number of **unique characters** in each string (**descending** order).
If two strings have the same number of unique characters, sort them **alphabetically**.

Assume that the characters in the words are only consisting of lower-case english alphabets

Function Definition (DO NOT CHANGE THIS NAME)

```
def sort_by_unique_chars(words):
    pass
```

check your code by running the below block

```
#example case
input_data = ["apple", "banana", "kiwi", "grape", "mango"]
output = sort_by_unique_chars(input_data)
print(output)
# Expected Output:
# ['grape', 'mango', 'apple', 'banana', 'kiwi']
```

Section 2 (NumPy)

Problem 1

In this problem, you will work with NumPy arrays to practice masking, slicing, advanced indexing, and broadcasting. Follow each step sequentially using only NumPy operations (no Python loops).

Problem Statement

1. **Generate** a 10×10 NumPy array `X` containing random integers between **0 and 100** (inclusive).
2. Create a **boolean mask** selecting all values between **20 and 50** (inclusive).
   Using this mask, replace those values in `X` with **–1** *in-place*.
3. Extract a **6×6 submatrix** `sub` from the modified `X`:

- Rows **2 to 8** (8 excluded)
- Columns **3 to 9** (9 excluded)

4. Using **advanced NumPy indexing**, extract all **diagonal elements** of the 6×6 submatrix `sub` into a 1D array `diag_vals`.
   Hint: use `np.arange(6)`.
5. Construct a **10×10 structured matrix** `M` using broadcasting, where:

   `M[i, j] = (i - j)²`

   for all `0 ≤ i, j < 10`. This must be done **without loops**.

Problem 2

In this problem, you will work with NumPy arrays representing class scores for multiple samples. You will practice row-wise normalization, broadcasting, and boolean masking.

Problem Statement

1. Create a NumPy array `scores` of shape 5×4 containing integer values between **0 and 20** (inclusive).
   Each row represents a sample, and each column represents a score for one of the 4 classes.
2. For each row in `scores`, subtract the **maximum value of that row** from all elements in that row.
   This operation must be performed using **broadcasting** (no loops allowed).

   Hint:
   `scores.max(axis=1, keepdims=True)` produces a (5×1) column of row-wise maxima.
3. Compute a new array `exp_scores` by applying the exponential function to each element:

   `exp_scores = np.exp(shifted_scores)`
4. **Normalize each row** of `exp_scores` so that each row sums to **1**.
   Store the resulting array in `probs`.

   The transformation is conceptually shown by the formula:

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

   Which corresponds to:

   `probs[i, j] = exp_scores[i, j]/sum(exp_scores[i, :])`
5. Compute the **predicted class** for each sample by taking the index of the largest value in each row of `probs`.
   Store this in a 1D array `y_pred` of length 5.
6. Create a NumPy array `y_true` of length 5 containing the true class labels (each between 0 and 3).
7. Create a boolean array `correct_mask` indicating whether each predicted label matches the true label.
   Then compute the **accuracy** using:

   `accuracy = correct_mask.mean()`

Section 3 - Pandas and MatPlotLib

Creating Batches for Training and Testing Using Pandas

Download the following .csv file from the given link directly using commands in colab - url = "https://raw.githubusercontent.com/rashida048/Datasets/master/StudentsPerformance.csv"

```
Start coding or generate with AI.
```

```
--2025-12-06 13:34:08--  https://raw.githubusercontent.com/rashida048/Datasets/master/StudentsPerformance.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 57021 (56K) [text/plain]
```

```
Saving to: ‘StudentsPerformance.csv’

StudentsPerformance 100%[===================>]  55.68K  --.-KB/s    in 0.02s

2025-12-06 13:34:08 (3.20 MB/s) - ‘StudentsPerformance.csv’ saved [57021/57021]
```

```
import pandas as pd
```

Import the csv file as a dataframe

```
df = pd.read_csv("StudentsPerformance.csv")
```

Write the command to be able to see the first 5 rows

```
Start coding or generate with AI.
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44

Create multiple dataframes with the following columns sorted : gender, race, math, reading and writing scores.

(meaning create df1, df2, df3 etc. with the math score sorted in df1, gender wise sorted in df2 etc.)

```
# gender
```

```
# race/ethnicity
```

```
# math
```

```
# reading
```

```
# writing
```

Create 2 non-overlapping dataframes test_df and train_df such that 20% of the rows are in test_df and the rest in train_df

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

Plot a bar graph such that you can see the distribution of race in test and train dataset. Are the proportions of races almost same in train and test datasets ? If not what can you do so that the proportions of races in the test and train datasets are close ?

Hint : Think Sorting on the race/ethnicity column and then applying some logic.

```
# to clarify if the split of races A, B, C, D and E are nearly say 0.2, 0.15, 0.25, 0.17, 0.23
# then I expect an almost similar split in the train_df and test_df
```

```
Start coding or generate with AI.
```

Even if it the proportions are almost same think of a way to create the train_df and test_df to have similar proportions of races.

```
Start coding or generate with AI.
```

Usually when we train machine learning models we use batches. Each batch is a subset of train_df of legth batch_size. Create Batches: a list of batch each of size 50 from the train_df

```
batch_size = 50
```

```
Start coding or generate with AI.
```

Print the size of Batches and first few rows of Batches[0]

```
Start coding or generate with AI.
```